HDOC - 73

Enumerations in C: Days of the Week and Traffic Lights

Question 1: Enumeration for Days

Create an enumeration for days (SUNDAY to SATURDAY) and print each day with its integer value. Expected values: SUNDAY = 0 through SATURDAY = 6.

Algorithm / Logic

1. Define an enum named Day containing SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, and SATURDAY.

2. Since no explicit values are assigned, C automatically assigns consecutive integer values starting from 0.

3. Store the day names in a string array in the same order.

4. Use a loop from SUNDAY to SATURDAY and print each name with its enum integer value.

C Program

```
#include <stdio.h>

enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY
};

int main(void) {
    const char *dayNames[] = {
        "SUNDAY", "MONDAY", "TUESDAY", "WEDNESDAY",
        "THURSDAY", "FRIDAY", "SATURDAY"
    };

    for (enum Day day = SUNDAY; day <= SATURDAY; day++) {
        printf("%s = %d", dayNames[day], day);
        if (day < SATURDAY)
            printf(", ");
    }
    printf("\n");
    return 0;
}
```

Explanation

An enumeration gives readable names to integer constants. Here, SUNDAY receives 0 automatically and every following identifier increases by 1, so SATURDAY becomes 6. The loop uses the enum values as indexes into the dayNames array.

Test Cases

Test	Input	Expected Output
1	No input	SUNDAY = 0, MONDAY = 1, ..., SATURDAY = 6

Sample Output

```
SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6
```

Result

The program successfully creates an enumeration for all seven days and prints each day with its corresponding integer value from 0 to 6.

Question 2: Enumeration for Traffic Lights

Define an enumeration for traffic lights (RED, YELLOW, GREEN) and print 'Stop', 'Wait', or 'Go' based on its value. Given input: GREEN. Expected output: Go.

Algorithm / Logic

1. Define an enum named TrafficLight containing RED, YELLOW, and GREEN.

2. Read the traffic-light state as text.

3. Convert the entered state to the corresponding enum value.

4. Use a switch statement on the enum value: RED -> Stop, YELLOW -> Wait, GREEN -> Go.

5. If the input does not match any valid light, display an invalid-input message.

C Program

```
#include <stdio.h>
#include <string.h>

enum TrafficLight { RED, YELLOW, GREEN };

int main(void) {
    char input[20];
    enum TrafficLight light;

    printf("Enter traffic light (RED/YELLOW/GREEN): ");
    scanf("%19s", input);

    if (strcmp(input, "RED") == 0)
        light = RED;
    else if (strcmp(input, "YELLOW") == 0)
        light = YELLOW;
    else if (strcmp(input, "GREEN") == 0)
        light = GREEN;
    else {
        printf("Invalid traffic light\n");
        return 1;
    }

    switch (light) {
        case RED:    printf("Stop\n"); break;
        case YELLOW: printf("Wait\n"); break;
        case GREEN:  printf("Go\n");   break;
    }

    return 0;
}
```

Explanation

The enum TrafficLight represents the three possible signal states using named integer constants. The entered word is matched using strcmp(), then a switch statement selects the correct instruction. This keeps the program readable and avoids using unexplained numeric values.

Test Cases

Test	Input	Expected Output

1	RED	Stop
2	YELLOW	Wait
3	GREEN	Go
4	BLUE	Invalid traffic light

Sample Output

```
Enter traffic light (RED/YELLOW/GREEN): GREEN
Go
```

Result

The program successfully uses an enumeration to represent traffic-light states and displays the appropriate action. For the input GREEN, the output is 'Go'.